

## Spring Core\_Maven

### Exercise 1: Configuring a Basic Spring Application

#### Scenario:

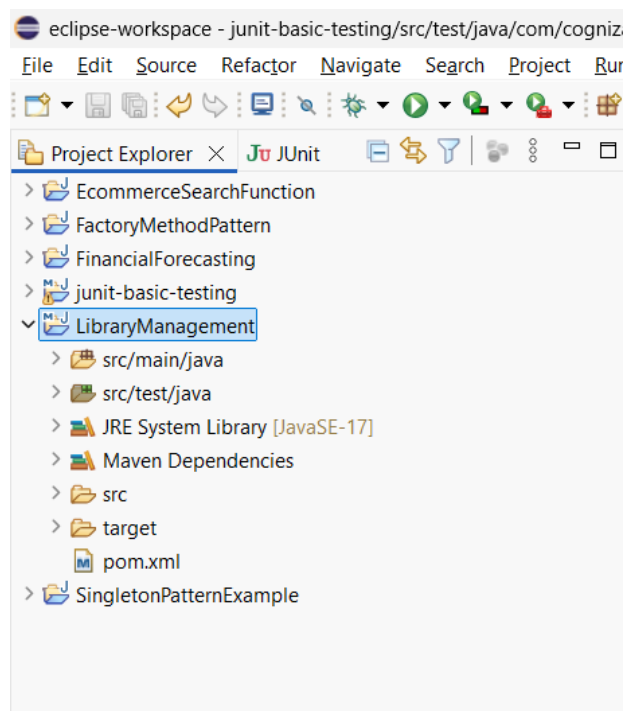
Your company is developing a web application for managing a library. You need to use the Spring Framework to handle the backend operations.

#### Steps:

1. **Set Up a Spring Project:**
  - Create a Maven project named **LibraryManagement**.
  - Add Spring Core dependencies in the **pom.xml** file.
2. **Configure the Application Context:**
  - Create an XML configuration file named **applicationContext.xml** in the **src/main/resources** directory.
  - Define beans for **BookService** and **BookRepository** in the XML file.
3. **Define Service and Repository Classes:**
  - Create a package **com.library.service** and add a class **BookService**.
  - Create a package **com.library.repository** and add a class **BookRepository**.
4. **Run the Application:**
  - Create a main class to load the Spring context and test the configuration.

#### Step 1: Open the Existing Maven Project

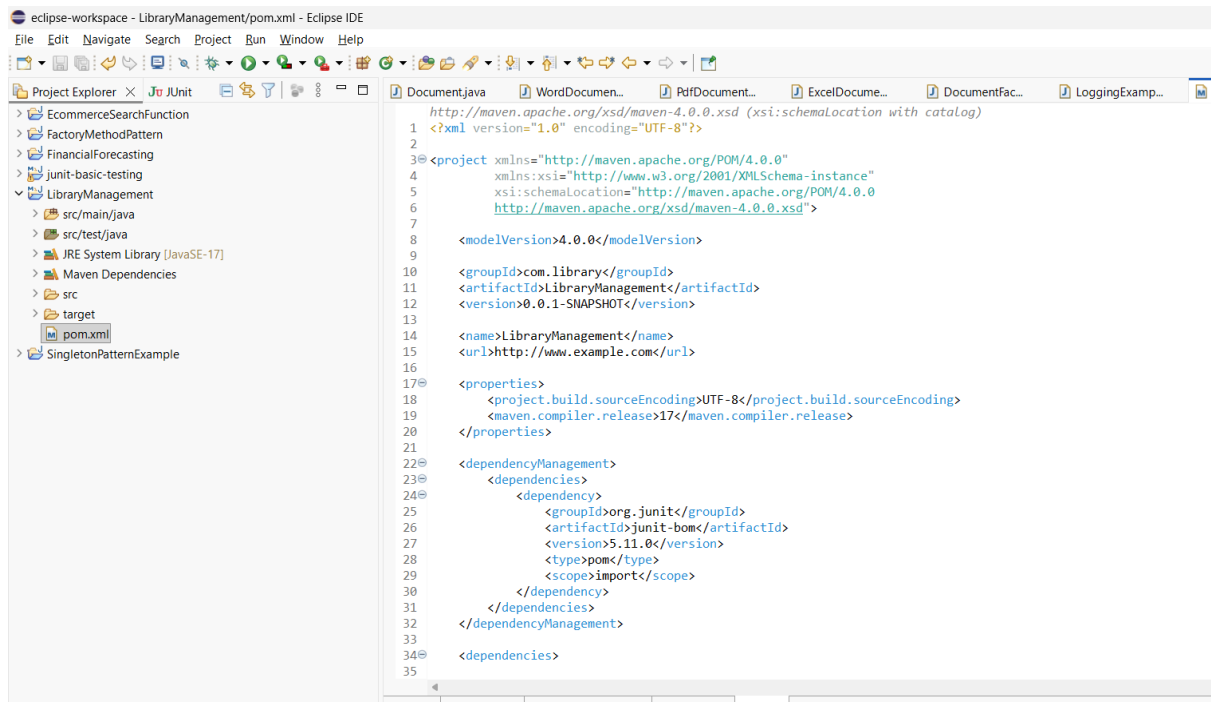
Opened the existing Maven project junit-basic-testing in Eclipse IDE.



# COGNIZANT DIGITAL NURTURE 5.0 – JAVA FSE

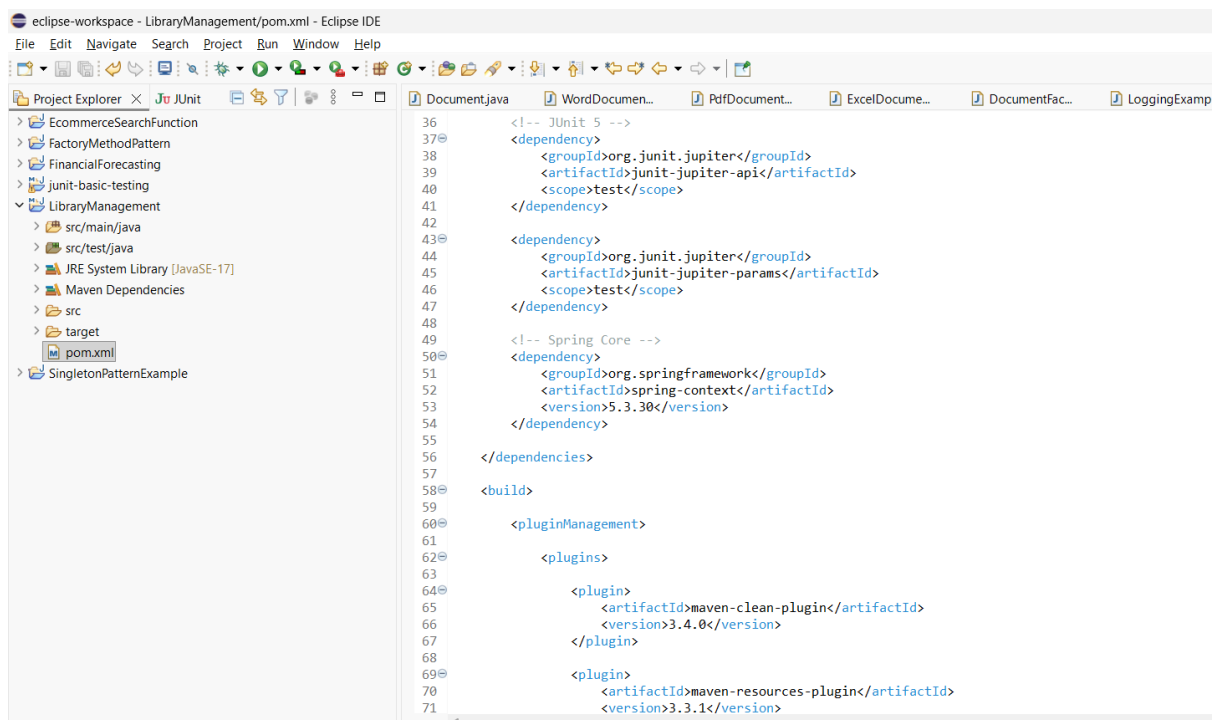
## Step 2: Open pom.xml

Opened the pom.xml file and added the required Spring Core dependency to enable Spring Framework support in the project.



The screenshot shows the Eclipse IDE with the 'pom.xml' file open. The Project Explorer on the left shows the project structure, including 'LibraryManagement' and 'SingletonPatternExample'. The main editor displays the XML content of 'pom.xml'.

```
1 http://maven.apache.org/xsd/maven-4.0.0.xsd (xsi:schemaLocation with catalog)
2
3 <?xml version="1.0" encoding="UTF-8"?>
4
5 <project xmlns="http://maven.apache.org/POM/4.0.0"
6   xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
7   xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
8     http://maven.apache.org/xsd/maven-4.0.0.xsd">
9
10  <modelVersion>4.0.0</modelVersion>
11
12  <groupId>com.library</groupId>
13  <artifactId>LibraryManagement</artifactId>
14  <version>0.0.1-SNAPSHOT</version>
15
16  <name>LibraryManagement</name>
17  <url>http://www.example.com</url>
18
19  <properties>
20    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
21    <maven.compiler.release>17</maven.compiler.release>
22  </properties>
23
24  <dependencyManagement>
25    <dependencies>
26      <dependency>
27        <groupId>org.junit</groupId>
28        <artifactId>junit-bom</artifactId>
29        <version>5.11.0</version>
30        <type>pom</type>
31        <scope>import</scope>
32      </dependency>
33    </dependencies>
34  </dependencyManagement>
35
36  <dependencies>
```



The screenshot shows the Eclipse IDE with the 'pom.xml' file open, displaying the updated XML content. The new dependencies added are for JUnit 5, Spring Core, and Maven plugins.

```
36 <!-- JUnit 5 -->
37 <dependency>
38   <groupId>org.junit.jupiter</groupId>
39   <artifactId>junit-jupiter-api</artifactId>
40   <scope>test</scope>
41 </dependency>
42
43 <dependency>
44   <groupId>org.junit.jupiter</groupId>
45   <artifactId>junit-jupiter-params</artifactId>
46   <scope>test</scope>
47 </dependency>
48
49 <!-- Spring Core -->
50 <dependency>
51   <groupId>org.springframework</groupId>
52   <artifactId>spring-context</artifactId>
53   <version>5.3.30</version>
54 </dependency>
55
56 </dependencies>
57
58 <build>
59
60   <pluginManagement>
61
62     <plugins>
63
64       <plugin>
65         <artifactId>maven-clean-plugin</artifactId>
66         <version>3.4.0</version>
67       </plugin>
68
69       <plugin>
70         <artifactId>maven-resources-plugin</artifactId>
71         <version>3.3.1</version>
```

# COGNIZANT DIGITAL NURTURE 5.0 – JAVA FSE

```
eclipse-workspace - LibraryManagement/pom.xml - Eclipse IDE
File Edit Navigate Search Project Run Window Help

Project Explorer
> EcommerceSearchFunction
> FactoryMethodPattern
> FinancialForecasting
> junit-basic-testing
> LibraryManagement
  > src/main/java
  > src/test/java
  > JRE System Library [JavaSE-17]
  > Maven Dependencies
  > src
  > target
  pom.xml
> SingletonPatternExample

Document.java WordDocument... PdfDocument... ExcelDocume... DocumentFac...

69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103

<plugin>
  <artifactId>maven-resources-plugin</artifactId>
  <version>3.3.1</version>
</plugin>

<plugin>
  <artifactId>maven-compiler-plugin</artifactId>
  <version>3.13.0</version>
</plugin>

<plugin>
  <artifactId>maven-surefire-plugin</artifactId>
  <version>3.3.0</version>
</plugin>

<plugin>
  <artifactId>maven-jar-plugin</artifactId>
  <version>3.4.2</version>
</plugin>

<plugin>
  <artifactId>maven-install-plugin</artifactId>
  <version>3.1.2</version>
</plugin>

<plugin>
  <artifactId>maven-deploy-plugin</artifactId>
  <version>3.1.2</version>
</plugin>

<plugin>
  <artifactId>maven-site-plugin</artifactId>
  <version>3.12.1</version>
</plugin>
```

```
eclipse-workspace - LibraryManagement/pom.xml - Eclipse IDE
File Edit Navigate Search Project Run Window Help

Project Explorer
> EcommerceSearchFunction
> FactoryMethodPattern
> FinancialForecasting
> junit-basic-testing
> LibraryManagement
  > src/main/java
  > src/test/java
  > JRE System Library [JavaSE-17]
  > Maven Dependencies
  > src
  > target
  pom.xml
> SingletonPatternExample

Document.java WordDocument... PdfDocument... ExcelDocume... DocumentFac... LoggingExamp...

80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115

<plugin>
  <artifactId>maven-surefire-plugin</artifactId>
  <version>3.3.0</version>
</plugin>

<plugin>
  <artifactId>maven-jar-plugin</artifactId>
  <version>3.4.2</version>
</plugin>

<plugin>
  <artifactId>maven-install-plugin</artifactId>
  <version>3.1.2</version>
</plugin>

<plugin>
  <artifactId>maven-deploy-plugin</artifactId>
  <version>3.1.2</version>
</plugin>

<plugin>
  <artifactId>maven-site-plugin</artifactId>
  <version>3.12.1</version>
</plugin>

<plugin>
  <artifactId>maven-project-info-reports-plugin</artifactId>
  <version>3.6.1</version>
</plugin>

</plugins>

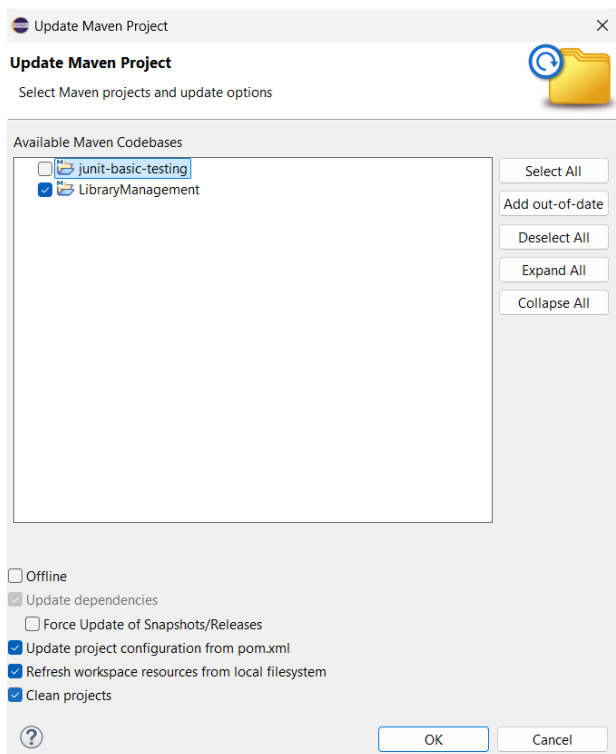
</pluginManagement>

</build>

</project>
```

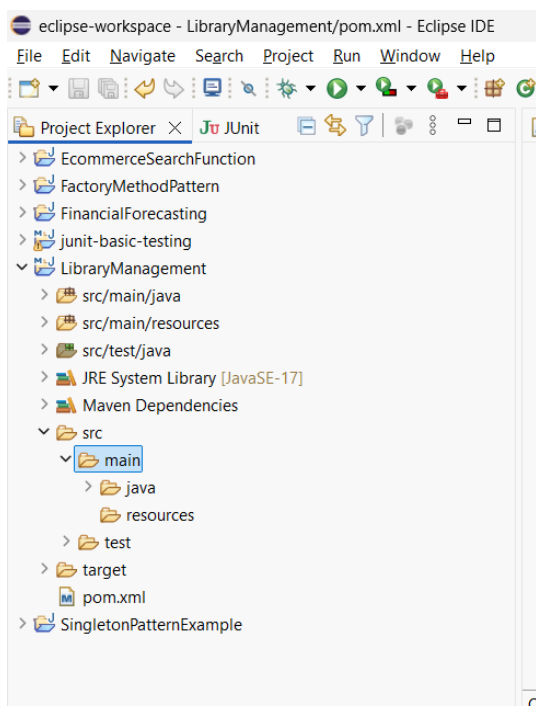
### Step 3: Update the Maven Project

Updated the Maven project to download all Spring dependencies and refresh the project configuration.



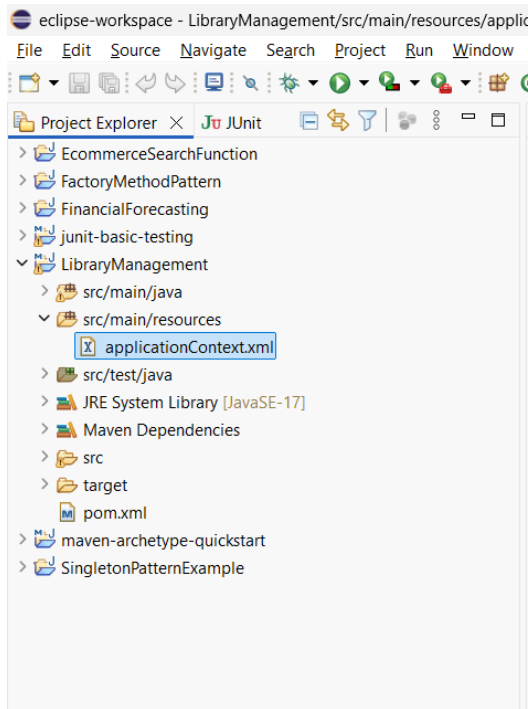
### Step 4: Verify the Project Structure

Verified that the Maven project contains the required directory structure, including src/main/java, src/main/resources, and pom.xml.



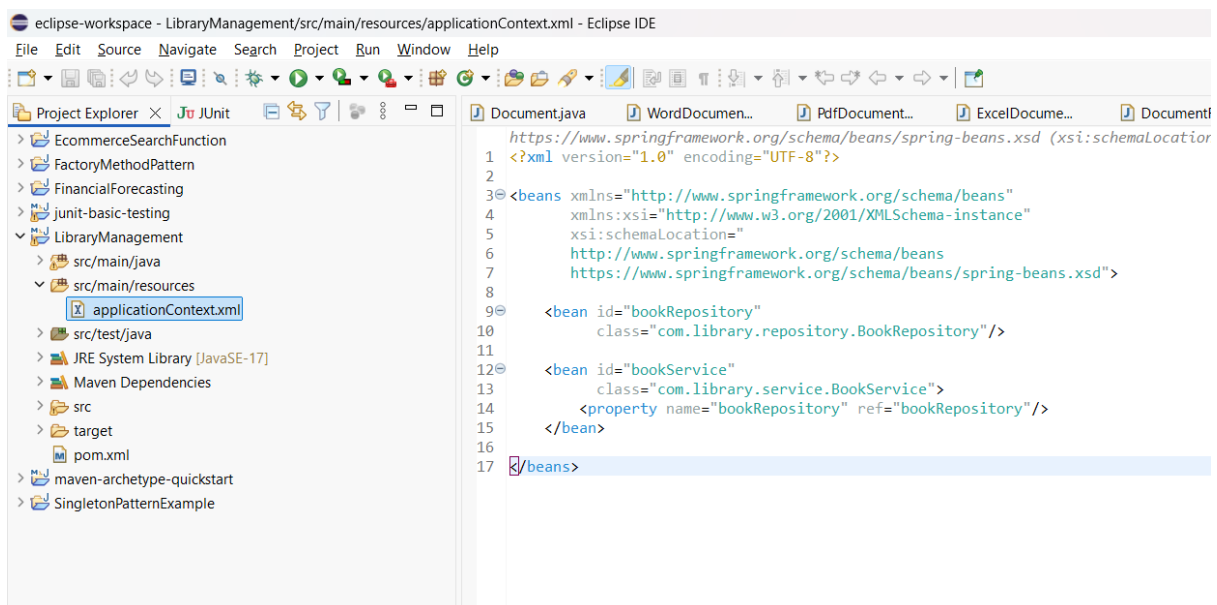
### Step 5: Create applicationContext.xml

Created the applicationContext.xml file inside the src/main/resources folder to configure Spring beans.



### Step 6: Configure Spring Beans

Configured the **BookRepository** and **BookService** beans in **applicationContext.xml** using XML-based Spring configuration.

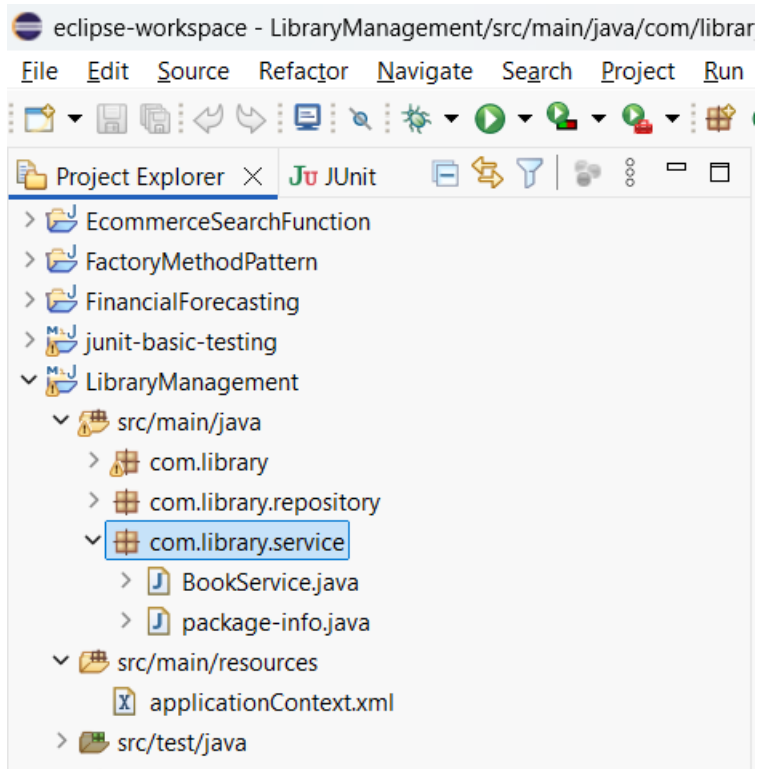
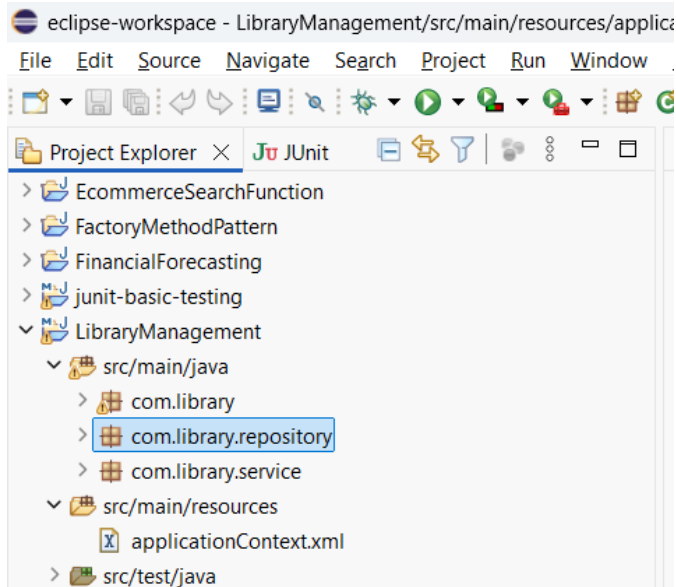


### Step 7: Create Java Packages

Created the required packages:

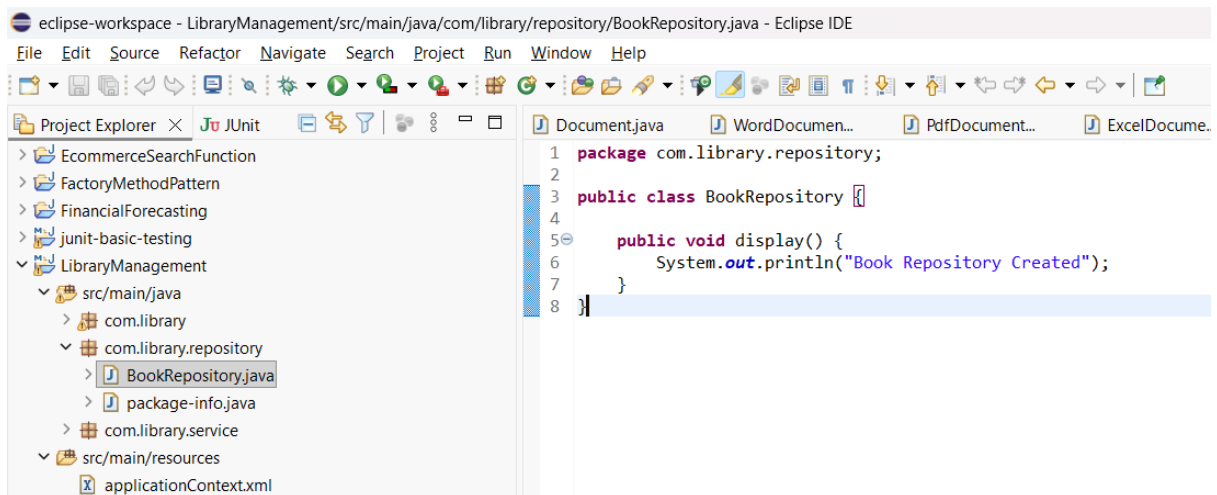
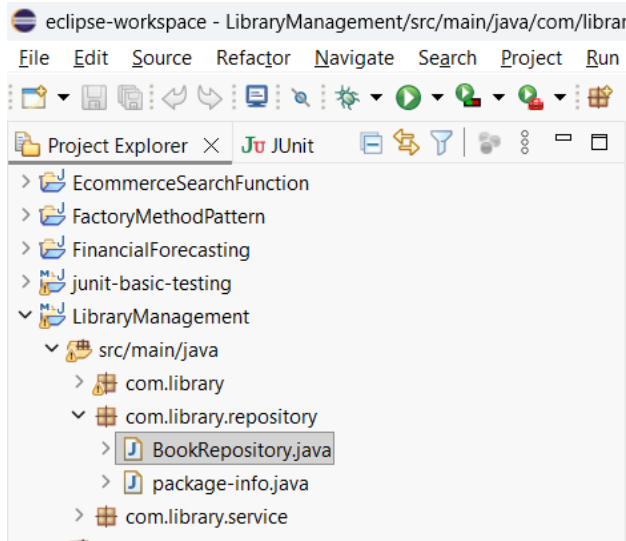
- **com.library.repository**
- **com.library.service**

to organize the application classes.



### Step 8: Create BookRepository Class

Implemented the **BookRepository** class, which represents the repository layer responsible for data-related operations.



### Step 9: Create BookService Class

Implemented the **BookService** class and injected the **BookRepository** bean using setter injection.

```
eclipse-workspace - LibraryManagement/src/main/java/com/library/service/BookService.java - Eclipse IDE
File Edit Source Refactor Navigate Search Project Run Window Help

Project Explorer
  > EcommerceSearchFunction
  > FactoryMethodPattern
  > FinancialForecasting
  > junit-basic-testing
  > LibraryManagement
    > src/main/java
      > com.library
      > com.library.repository
      > com.library.service
        > BookService.java
        > package-info.java
    > src/main/resources
      > applicationContext.xml
    > src/test/java
    > JRE System Library [JavaSE-17]
    > Maven Dependencies
    > src
    > target
    > pom.xml
    > maven-archetype-quickstart

Document.java WordDocument... PdfDocument... ExcelDocume... Docu...
1 package com.library.service;
2
3 import com.library.repository.BookRepository;
4
5 public class BookService {
6
7     private BookRepository bookRepository;
8
9     public void setBookRepository(BookRepository bookRepository) {
10         this.bookRepository = bookRepository;
11     }
12
13     public void displayService() {
14         System.out.println("Book Service Created");
15         bookRepository.display();
16     }
17 }
```

### Step 10: Create the Main Class

Created the **LibraryApplication** class to load the Spring Application Context and retrieve the configured bean.

```
eclipse-workspace - LibraryManagement/src/main/java/com/library/LibraryApplication.java - Eclipse IDE
File Edit Source Refactor Navigate Search Project Run Window Help

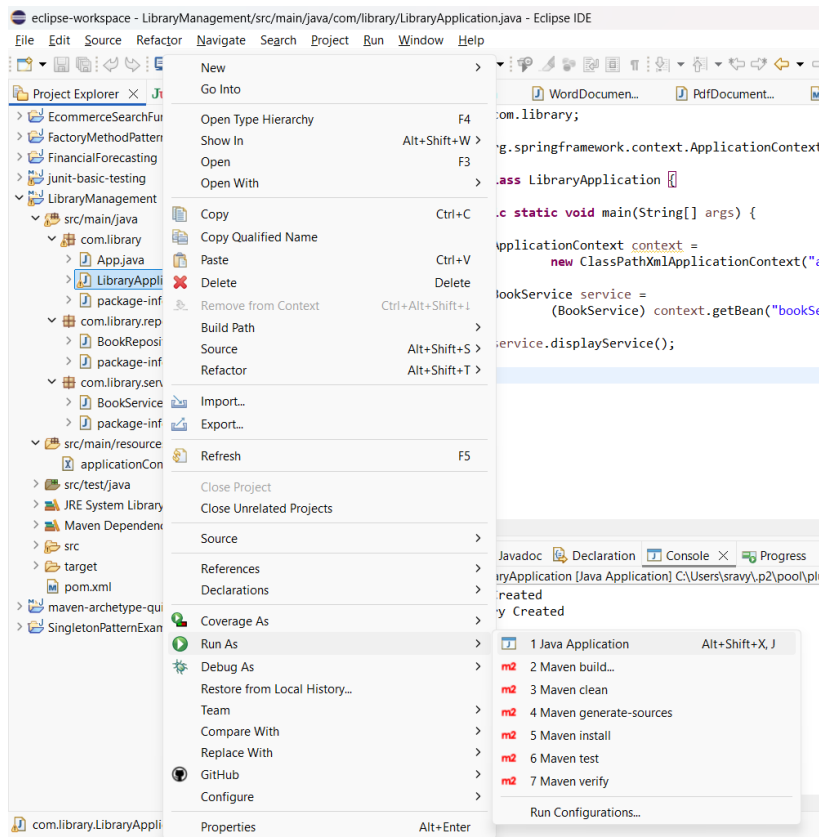
Project Explorer
  > EcommerceSearchFunction
  > FactoryMethodPattern
  > FinancialForecasting
  > junit-basic-testing
  > LibraryManagement
    > src/main/java
      > App.java
      > LibraryApplication.java
      > package-info.java
    > com.library.repository
      > BookRepository.java
      > package-info.java
    > com.library.service
      > BookService.java
      > package-info.java
    > src/main/resources
      > applicationContext.xml
    > src/test/java
    > JRE System Library [JavaSE-17]
    > Maven Dependencies
    > src
    > target
    > pom.xml
    > maven-archetype-quickstart
    > SingletonPatternExample

Document.java WordDocument... PdfDocument... LibraryManag... applicationC... BookReposito... BookService... LibraryAppli...
1 package com.library;
2
3 import org.springframework.context.ApplicationContext;
4
5 public class LibraryApplication {
6
7     public static void main(String[] args) {
8
9         ApplicationContext context =
10             new ClassPathXmlApplicationContext("applicationContext.xml");
11
12         BookService service =
13             (BookService) context.getBean("bookService");
14
15         service.displayService();
16     }
17 }
```



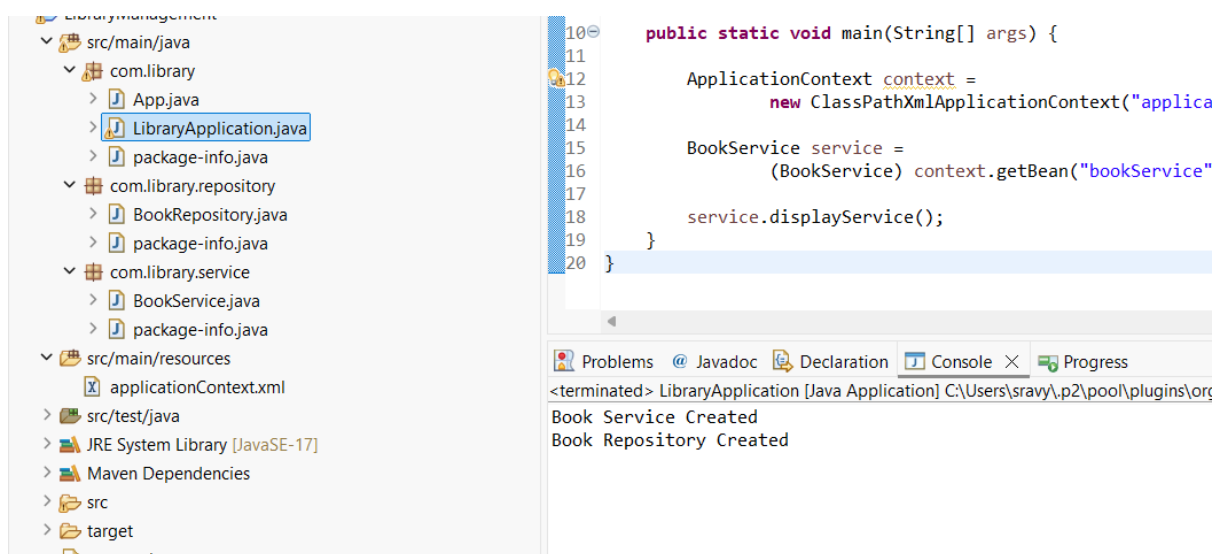
## Step 11: Execute the Application

Executed the **LibraryApplication.java** file as a Java Application to verify that the Spring configuration was successful.



## Output

The above output confirms that the Spring Application Context successfully created and managed the configured beans.



## COGNIZANT DIGITAL NURTURE 5.0 – JAVA FSE

### Result

The Library Management application was successfully configured using the Spring Framework and Maven. The application loaded the XML configuration file, created the **BookRepository** and **BookService** beans, injected the dependencies correctly, and executed successfully.